

第5章 内存管理

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 回顾栈/堆/静态区模型,对照 C 的 malloc/free 的痛点(泄漏、悬垂、重复释放)理解 GC 的意义
- 理解 Go GC 的三色标记并发回收原理:何时发生、停顿多短、怎么调 GOGC
- 掌握逃逸分析:变量**可能被编译器放到堆上**,用 `go build -gcflags=-m` 观察分配位置
- 掌握 defer 释放资源的惯用法与 sync.Pool 对象池的正确姿势
- 理解 GC 语言中内存泄漏的真实形态:**goroutine 泄漏**、全局容器、闭包误捕获
- 会用 pprof 采集与阅读内存画像,定位泄漏与高频分配
- 掌握分配性能实践:预分配容量、strings.Builder、避免热路径装箱

💡 从 C 到 Go:本章主题如何迁移

C 的内存管理是"全手动": `malloc / free` 成对出现,漏 free 是泄漏,free 两次是崩溃,free 后用是悬垂指针。每一个 C 老兵都经历过 valgrind 与 ASan 的深夜陪伴。Go 选择了另一条路:**垃圾回收(GC)**——程序员只管分配,回收交给运行时。

这不是偷懒,而是工程权衡:GC 换来的是**确定性**——不存在悬垂指针、use-after-free、重复释放;任何时刻访问任何 Go 指针都是安全的。代价是:回收不是即时的,内存峰值可能更高,且 GC 停顿(Stop The World)需要被控制在毫秒级。Go 用**三色标记 + 并发回收**把停顿压到亚毫秒,代价是 GC 期间 CPU 有额外开销。

于是 Go 程序员的"内存意识"从"什么时候 free"变成了三个新问题:**变量分配在栈还是堆?**(逃逸分析决定,与 C 的"局部变量必然在栈"不同);**谁还引用着这块内存?**(GC 语言的内存泄漏不是"忘记释放",而是"忘记解除引用"——goroutine 泄漏是头号形态);**分配频繁吗?**(GC 语言里,每秒百万次小分配会拖垮吞吐,对象池与预分配成为性能武器)。

一句话主线: C 手动管理内存,错误是"内存错误";Go 自动管理内存,错误是"引用错误"——两者排查工具也因此不同:前者 valgrind,后者 pprof。

📦 前置知识自查

- C 的 malloc/calloc/realloc/free,以及泄漏、悬垂指针、double free 三种经典错误
- 栈(函数调用帧)与堆(动态分配)的基本模型
- 指针与数组的关系;函数返回局部变量地址的危险
- 第 4 章的 defer 与闭包捕获语义(本章直接使用)

🚀 本章学习路线

1. **第一阶段**:§5.1–§5.2,建立 GC 心智模型:为什么 Go 敢说"没有内存错误"
2. **第二阶段**:§5.3–§5.5,动手跑逃逸分析命令,复现 goroutine 泄漏并修复——这是本章最重要的实操
3. **第三阶段(实验)**:§5.6–§5.7 与章末实验:内存泄漏复现与 pprof 定位修复
4. **验收标准**:能说出 GC 语言内存泄漏的四种形态,能跑出 `-gcflags=-m` 的 "moved to heap" 输出

本章知识导图

- §5.1 从 malloc/free 到 GC:三种内存错误的消失
- §5.2 GC 原理:三色标记、并发回收、STW 与 GOGC
- §5.3 逃逸分析:变量在哪分配? `-gcflags=-m` 观察
- §5.4 defer 资源释放与 sync.Pool 对象池
- §5.5 内存泄漏的 Go 形态:goroutine 泄漏、全局容器、闭包
- §5.6 pprof 内存画像:采集与阅读
- §5.7 分配性能:预分配、strings.Builder、热路径

§5.1 从 malloc/free 到 GC:三种内存错误的消失

C 的内存错误谱系:泄漏(忘记 free,内存持续增长)、悬垂指针(free 后仍访问,行为未定义)、重复释放(double free,堆损坏)。Go 用 GC 一揽子消灭:

C 的写法	Go 的写法
<pre>int* p = malloc(n); ...; free(p);</pre>	<pre>p := make([]int, n),不需要释放</pre>
忘记 free → 泄漏;多次 free → 崩溃	不可达对象自动回收;不存在"错误释放"
free 后访问 → UB	只要还被引用就仍有效;引用消失才被回收
valgrind / ASan 检测内存错误	无需此类工具;转而用 pprof 查"谁还引用着"

代价清单也要说清楚:GC 不是免费的。①**内存峰值**:对象不会立刻被回收,堆用量可能高于"精确释放"方案;②**CPU 开销**:标记与清扫本身消耗计算资源;③**停顿**:虽然并发,仍有极短的全局停顿。因此 Go 程序不是"随便造对象都行"——**分配量仍然是最重要的性能指标之一**。

§5.2 GC 原理:三色标记、并发回收、GOGC

Go 的 GC 是**并发三色标记-清除**(CMS 思路)。三个概念:

- **三色标记**:对象分白(未被访问)、灰(已访问,其引用未处理)、黑(已处理)。从根集合(全局变量、栈)出发,灰→黑推进,引用的白→灰;结束时仍为白的对象即垃圾,被清除
- **并发回收**:标记过程与用户代码并发执行(写屏障保证正确性),只有极短的 STW 窗口;从 Go 1.5 的"毫秒级停顿"到近几代版本的"亚毫秒级"

- **GOGC**:触发阈值,默认 100——堆增长 100% 时触发 GC。调大(如 `GOGC=400`)减少 GC 频率、抬高内存峰值;调小(`GOGC=20`)降低峰值、增加 CPU 开销。压测时用 `GODEBUG=gctrace=1` 观察每次 GC 的耗时与堆大小

★ 重点:GC 调参三句话

①默认 `GOGC=100` 在绝大多数场景已经最优,先别动;②内存吃紧(容器限制)时调小,追求吞吐时调大;③Go 1.19+ 的 `debug.SetMemoryLimit` 可以给堆设硬上限,比反复调 GOGC 更直观: `debug.SetMemoryLimit(512 << 20)`。生产上建议结合压测数据决定,不要拍脑袋。

§5.3 逃逸分析:变量分配在哪?

C 的规则简单:局部变量在栈, `malloc` 的在堆,函数返回局部变量地址是危险行为。Go 没有这么直白——**编译器自动决定**变量放哪,依据是"变量是否会逃逸出当前函数"。

示例 5-1 逃逸分析:小对象也可能在堆上

```
package main

import "fmt"

// makeCounter 返回闭包:捕获的 n 逃逸到堆上
// 用 go build -gcflags=-m 观察输出 "moved to heap: n"
func makeCounter() func() int {
    n := 0
    return func() int {
        n++
        return n
    }
}

// add 内联 + 栈分配:参数与结果都不逃逸
func add(a, b int) int {
    return a + b
}

func main() {
    c := makeCounter()
    fmt.Println(c(), c(), c())
    fmt.Println(add(1, 2))
}
```

示例 5-2 观察逃逸:go build -gcflags=-m

在命令行执行:

```
go build -gcflags=-m main.go
# 关键输出(节选):
# ./main.go:12:6: moved to heap: n          ← 闭包捕获的 n 分配在堆上
# ./main.go:18:6: can inline add          ← add 被内联,无分配
```

解读:"moved to heap" 不是坏事,是编译器在保证安全:闭包返回后 `n` 必须存活,放堆是唯一正确选择(这正是 C 里"返回局部变量地址"的 UB 场景!). 你不需要手动干预,但要知道**哪些写法会产生堆分配**:闭包捕获、接口装箱、指针逃逸、大结构体。

易错点 5-1:以为"局部变量一定在栈上"

Go 没有 `malloc / free` 语法,初学者以为"没写分配就是栈"。实际:闭包捕获的变量、被取地址并返回的变量、存进接口的变量,都可能被移到堆。判断工具: `go build -gcflags=-m` 与 `pprof` 的 `alloc_objects` 视图。这条知识是"为什么闭包能安全返回"的底层答案。

§5.4 defer 资源释放与 sync.Pool

GC 管理内存,但**不管理所有资源**:文件句柄、网络连接、锁、数据库事务都需要显式释放。Go 的惯用法是第 4 章的 `defer` ——保证函数退出时一定释放,无论中间是否出错。

示例 5-3 defer 保证文件释放

```
package main

import (
    "fmt"
    "os"
)

func writeLog(path, msg string) error {
    f, err := os.OpenFile(path, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
    if err != nil {
        return err
    }
    defer f.Close() // 无论 writeString 成败,句柄必定释放

    _, err = f.WriteString(msg + "\n")
    return err
}

func main() {
    if err := writeLog("app.log", "服务启动"); err != nil {
        fmt.Println("写日志失败:", err)
    }
}
```

sync.Pool 是"对象池":存放临时对象,Get 时若无则用 New 创建,Put 归还。它适合**高频创建、可复用、无状态残留**的对象(如缓冲区)。注意两个陷阱:池可能随时被 GC 清空(它是缓存不是长期存储);取出后必须重置状态。

示例 5-4 sync.Pool 复用缓冲区

```
package main

import (
    "fmt"
    "sync"
)

var bufPool = sync.Pool{
    New: func() any {
        return make([]byte, 0, 4096) // 默认容量 4KB
    },
}

// render 从池中取缓冲区,用后归还
func render(n int) string {
    buf := bufPool.Get().([]byte)
    defer func() {
        buf = buf[:0] // 重置长度,防止数据残留
        bufPool.Put(buf)
    }()
    for i := 0; i < n; i++ {
        buf = append(buf, 'x')
    }
    return string(buf) // string 会拷贝,返回后缓冲区可安全复用
}

func main() {
    for i := 0; i < 5; i++ {
        fmt.Println(len(render(100)))
    }
}
```

⚠ 易错点 5-2:sync.Pool 不是缓存!别指望它"长期复用"

Pool 中的对象在每次 GC 时可能被整体丢弃(这是设计:GC 语言里对象池的职责是降低同一时间窗口的分配峰值,不是跨窗口保存)。因此:①不要用 Pool 存有状态的重要对象;②Get 后的对象可能是新创建的,必须初始化;③放回前重置长度/清空内容,否则下一个使用者读到残留数据。

§5.5 内存泄漏的 Go 形态:goroutine 泄漏是头号

GC 语言的内存泄漏不是"忘了释放",而是**对象被持续引用,永远可达**。Go 生产环境的四大形态:

形态	成因	典型场景
goroutine 泄漏(头号)	goroutine 阻塞在 channel 上,无人关闭/无人接收,永不退出	任务队列无消费者、超时未设置
全局容器只增不减	map/切片存入全局缓存,从不淘汰	用户会话缓存、去重集合
闭包误捕获	长生命周期对象持有短生命周期对象的引用	回调注册后不注销;事件订阅
定时器未停止	time.After/time.NewTicker 未释放,关联对象被钉住	循环里 time.After、长期 ticker

示例 5-5 goroutine 泄漏:复现与修复

```

package main

import (
    "fmt"
    "time"
)

// 泄漏版:worker 阻塞在接收上,永远不退出
func leakyWorker(jobs <-chan int) {
    for j := range jobs {
        fmt.Println("处理", j)
    }
    // 缺陷:jobs 永远不关闭,此 goroutine 永久挂起
}

// 修复版:加超时退出(或由调用方显式关闭)
func safeWorker(jobs <-chan int, quit <-chan struct{}) {
    for {
        select {
        case j, ok := <-jobs:
            if !ok {
                return // 通道关闭,正常退出
            }
            fmt.Println("处理", j)
        case <-quit:
            return // 收到退出信号
        case <-time.After(3 * time.Second):
            fmt.Println("3 秒无任务,自动退出")
            return
        }
    }
}

func main() {
    jobs := make(chan int)
    quit := make(chan struct{})
    go safeWorker(jobs, quit)

    jobs <- 1
    jobs <- 2
    time.Sleep(4 * time.Second) // 超时后 worker 自动退出
    fmt.Println("main 结束,无泄漏 goroutine")
}

```

★ 重点:goroutine 泄漏为什么可怕

每个挂起的 goroutine 至少占 2KB 栈,还会钉住其闭包捕获的所有对象——一次"任务队列没有消费者"的事故,一天可以泄漏几百 MB。修复原则:**每个 goroutine 都要有退出路径**:要么通道被关闭,要么有超时/取消(第 9 章 context 是标准答案)。压测时用 `runtime.NumGoroutine()` 监控 goroutine 数是否随时间增长。

§5.6 pprof 内存画像:采集与阅读

Go 的官方性能剖析工具是 `net/http/pprof` (Web 服务) 与 `runtime/pprof` (CLI 程序)。内存分析两个视图: `inuse_space` (当前占用) 与 `alloc_objects` (累计分配次数)——**找泄漏看 inuse, 找分配热点看 alloc**。

示例 5-6 引入 pprof 采集端点

```
package main

import (
    "fmt"
    "net/http"
    _ "net/http/pprof" // 空白导入:注册 /debug/pprof/ 路由
)

func main() {
    fmt.Println("pprof 已启动: http://localhost:6060/debug/pprof/")
    // 业务服务挂在同一 mux 上即可;下面是演示用监听
    _ = http.ListenAndServe("localhost:6060", nil)
}
```

示例 5-7 pprof 命令行三板斧

```
# 1. 实时内存占用 top(单位换算成 MB)
go tool pprof -top -unit=MB http://localhost:6060/debug/pprof/heap

# 2. 定位"谁分配了最多对象"(分配热点)
go tool pprof -top -sample_index=alloc_objects http://localhost:6060/debug/pprof/heap

# 3. 交互式浏览分配调用链(web 输出火焰图)
go tool pprof http://localhost:6060/debug/pprof/heap
# 交互界面输入:top / list 函数名 / web
```

解读输出:每个条目显示函数、当前 `inuse` 内存、占比。若某个函数的 `inuse` 持续增长且调用链无法回退,基本就是泄漏点;`alloc_objects` 高但 `inuse` 低,说明是高频短命分配,优化方向是复用(预分配、对象池)。

§5.7 分配性能:预分配与 `strings.Builder`

GC 语言里"分配"本身有成本(分配器 + 后续回收)。三件小事能省下一大截:

示例 5-8 字符串拼接:+= 与 `strings.Builder`

```

package main

import (
    "fmt"
    "strings"
)

// 慢:每次 += 都创建新字符串,旧字符串留给 GC
func slowJoin(n int) string {
    s := ""
    for i := 0; i < n; i++ {
        s += "x"
    }
    return s
}

// 快:可变缓冲区 + 预分配容量
func fastJoin(n int) string {
    var b strings.Builder
    b.Grow(n) // 预分配,避免反复扩容
    for i := 0; i < n; i++ {
        b.WriteByte('x')
    }
    return b.String()
}

func main() {
    fmt.Println(len(slowJoin(10000)), len(fastJoin(10000)))
}

```

示例 5-9 切片预分配容量

```

package main

import "fmt"

// 不预分配:append 反复扩容+拷贝
func collectNoCap(n int) []int {
    var out []int
    for i := 0; i < n; i++ {
        out = append(out, i)
    }
    return out
}

// 预分配:一次分配到位
func collectCap(n int) []int {
    out := make([]int, 0, n) // 容量一步到位
    for i := 0; i < n; i++ {
        out = append(out, i)
    }
    return out
}

func main() {
    fmt.Println(len(collectNoCap(1000)), len(collectCap(1000)))
}

```



工程建议:热路径的分配纪律

①能预分配就预分配(make 带容量、Grow);②字符串拼接一律 strings.Builder,别用 += ;③接口装箱 (interface{})与 fmt.Sprintf 会分配,循环里改用 strconv;④用 go test -bench=. -benchmem 量化每次改动, alloc/op 是最诚实的指标;⑤先用 pprof 定位再优化——盲目优化分配次数是浪费生命。

本章速查表

主题	要点
内存模型	栈(函数帧)/堆(动态)/静态区(全局);GC 只管理堆
GC 原理	并发三色标记-清除;白/灰/黑三态;写屏障;极短 STW
GOGC	默认 100(堆翻倍触发);调小省内存、调大省 CPU;1.19+ 用 SetMemoryLimit
逃逸分析	编译器决定栈/堆;观察命令 <code>go build -gcflags=-m</code>
必然堆分配	闭包捕获、返回指针、接口装箱、大对象
资源释放	<code>defer Close()/Unlock()</code> ;GC 不管句柄/连接/锁
<code>sync.Pool</code>	临时对象池;Get 后要初始化,Put 前要重置;GC 会清空
泄漏形态	goroutine 挂起、全局容器、闭包引用、定时器未停
goroutine 泄漏	每个 goroutine 必须有退出路径:关闭/超时/取消
<code>pprof</code>	<code>net/http/pprof</code> 采集; <code>inuse_space</code> 找泄漏, <code>alloc_objects</code> 找热点
字符串拼接	<code>strings.Builder + Grow</code> ;循环内禁用 <code>+=</code>
预分配	<code>make([]T, 0, n)</code> ;map 预分配容量;append 扩容有拷贝
基准测试	<code>go test -bench=. -benchmem</code> 看 alloc/op

最小必会清单

- 能说出 Go 消灭了 C 的哪三类内存错误,以及 GC 的三种代价
- 能讲出三色标记的基本流程与"白=垃圾"的判断
- 能说出 GOGC 的含义,以及调大/调小各自代价
- 能跑出 `go build -gcflags=-m` 并解释 "moved to heap"
- 能写出"defer 关闭文件/解锁"与"`sync.Pool` 取出重置、归还清零"的完整代码
- 能说出 GC 语言内存泄漏的四种形态,并解释 goroutine 泄漏的机制
- 能写出带超时/退出信号的 worker 循环(select 三路)
- 能说出 `pprof` 两个采样视图分别用于什么场景

自测题

Q 1. 下面的函数返回后, `n` 在哪里?为什么? `func f() func() int { n := 0; return func() int { n++; return n } }`

✅ 参考答案

在堆上。闭包在函数返回后仍需要访问 `n`,编译器通过逃逸分析把它 "moved to heap"。这等价于 C 中"返回局部变量地址"的 UB 场景,但 Go 用堆分配把它变成安全行为。

Q 2. 判断正误:(a) Go 有 GC,所以不会内存泄漏;(b) 有 GC 就不需要手动释放文件句柄;(c) GOGC 越大,内存峰值越高;(d) sync.Pool 可以当长期缓存用。

✓ 参考答案

(a) 错,GC 语言一样泄漏(goroutine 泄漏、全局容器);(b) 错,GC 只管理内存,文件/连接/锁要 defer 释放;(c) 对,GOGC 大 = 更久才触发 GC = 堆更大;(d) 错,Pool 会被 GC 清空,只适合短期复用。

Q 3. 一段代码每秒处理 10 万条消息,每条都要拼接字符串。写出最优拼接方案,并解释为什么。

✓ 参考答案

用 `strings.Builder` 并 `Grow` 预分配(每条消息的缓冲区可以来自 `sync.Pool`)。理由: `+=` 每次创建新字符串, $O(n^2)$ 分配; `strings.Join` 也 $O(n)$,但 `Builder` 可以复用缓冲区并把分配次数降到常数。

Q 4. 生产服务内存持续增长但 `pprof inuse_space` 没有明显热点函数,下一步怎么查?

✓ 参考答案

① 先确认是堆增长还是系统内存:看 `runtime.MemStats` 的 `HeapInuse` 与 `Sys`;② 查 `goroutine` 数(`/debug/pprof/goroutine`),增长则定位 `goroutine` 泄漏;③查全局容器/缓存的容量与淘汰策略;④查定时器与 `ticker` 是否未 `Stop`;⑤对比两次 `heap dump` 的差异。

Q 5. GOGC=100 时,某时刻堆使用 100MB,下一次 GC 大约什么时候触发?

✓ 参考答案

堆增长到约 200MB 时触发(翻倍)。此后以"上次 GC 后的存活量为基准"继续:GC 结束后存活 120MB,则下次在约 240MB 触发。想限制峰值可用 `debug.SetMemoryLimit`。

Q 6. 为什么说"goroutine 泄漏是 Go 的头号内存泄漏"?修复的核心原则是什么?

✓ 参考答案

因为它是"代码写得完全合法"却仍泄漏:goroutine 阻塞在 `channel` 上,没有语法错误,只有运行时无限挂起,且每个挂起 goroutine 钉住其栈与闭包捕获的对象。核心原则:每个 goroutine 必须有退出路径——通道关闭、超时、`context` 取消(第 9 章)。

章末实验题:内存泄漏复现与修复

实验 5:内存泄漏复现、定位与修复

实验目标:构造一个"每个请求启动一个永不退出的 goroutine + 全局缓存只增不减"的泄漏程序,用 pprof 定位,再修复,并用运行时指标验证。

实验要求:

- 任务 1:复现泄漏:无缓冲 channel + 永不关闭,worker goroutine 挂起(覆盖:\$5.5 goroutine 泄漏)
- 任务 2:用 `runtime.NumGoroutine()` 打印 goroutine 数,确认随请求数线性增长(覆盖:\$5.5 监控手段)
- 任务 3:用 `runtime.ReadMemStats` 观察 HeapObjects/HeapAlloc 增长(覆盖:\$5.5 内存指标)
- 任务 4:修复方案一:有缓冲 channel + 显式 close(覆盖:\$5.5 退出路径)
- 任务 5:修复方案二:select + 超时/退出信号(覆盖:\$5.4、\$5.5)
- 任务 6:修复全局缓存:LoadOrStore 去重或限制键数量(覆盖:\$5.5 全局容器)
- 任务 7:运行 `go run -race main.go` 与基准前后对比,并用 pprof 观察修复前后 inuse(覆盖:\$5.6)

实验环境:Go 1.21+;命令: `go run main.go`、`go vet`、`go tool pprof`。

第一步 写"错误实现": `jobs := make(chan int)` 无缓冲, `go worker(jobs)` 后只发任务不关闭;调用 1000 次观察 goroutine 数

第二步 `memStats()` 辅助函数,打印 goroutine 数与堆对象数

第三步 写"修复实现":缓冲 1 + 发送后 close + 等待 done;或 select 三路(任务/退出/超时)

第四步 局缓存改用 `sync.Map` + 键数量上限,超限先删除最旧的(简版淘汰)

第五步 测对比: `go test -bench=BenchmarkHandler -benchmem` (把 handler 抽成可测函数)

✔ 参考答案要点

```

// 内存泄漏复现与修复实验
package main

import (
    "fmt"
    "runtime"
    "sync"
    "time"
)

// 全局缓存:泄漏点 1(只增不减)
var cache = sync.Map{}

// 错误实现:worker 挂在无缓冲 channel 上,永不退出(泄漏点 2)
func wrongHandler() {
    jobs := make(chan int)
    go func() {
        for j := range jobs {
            fmt.Println("job", j)
        }
    }()
    jobs <- 42 // 发送成功,但 jobs 永远不关闭
}

// 修复 1:缓冲 channel + 发送后关闭 + 等待退出
func fixedHandler() {
    jobs := make(chan int, 1) // 缓冲 1:发送永不阻塞
    done := make(chan struct{})
    go func() {
        defer close(done)
        for j := range jobs {
            fmt.Println("job", j)
        }
    }()
    jobs <- 42
    close(jobs) // 发送完立即关闭,worker 处理完就退出
    <-done      // 等 worker 真正退出
}

// 修复 2:select 三路,超时兜底(适合"无法确定何时关闭"的场景)
func timeoutHandler() {
    jobs := make(chan int, 1)
    go func() {
        for {
            select {
            case j, ok := <-jobs:
                if !ok {
                    return
                }
                fmt.Println("job", j)
            case <-time.After(2 * time.Second): // 空闲超时退出
                fmt.Println("空闲超时,退出")
                return
            }
        }
    }()
}

```

```

}()
jobs <- 42
time.Sleep(3 * time.Second) // 触发超时退出
}

// 修复 3:全局缓存限制键数量(简单淘汰)
func cachePut(key string, val any) {
    if _, loaded := cache.LoadOrStore(key, val); !loaded {
        fmt.Println("缓存新增:", key)
    }
    // 生产应接入 LRU 等淘汰策略;此处仅演示"键数量受控"
    count := 0
    cache.Range(func(k, v any) bool {
        count++
        if count > 100 {
            cache.Delete(k) // 超过 100 个键就删(演示用简化策略)
        }
        return true
    })
}

// memStats 打印运行时内存与 goroutine 指标
func memStats(tag string) {
    var m runtime.MemStats
    runtime.ReadMemStats(&m)
    fmt.Printf("[%s] goroutine=%d 堆对象=%d 堆内存=%.1fMB\n",
        tag, runtime.NumGoroutine(), m.HeapObjects,
        float64(m.HeapAlloc)/1024/1024)
}

// runLeaky 泄漏版入口(实验对比用)
func runLeaky(n int) {
    for i := 0; i < n; i++ {
        wrongHandler() // 每个请求泄漏一个 goroutine
    }
    memStats("泄漏版")
}

// runFixed 修复版入口
func runFixed(n int) {
    for i := 0; i < n; i++ {
        fixedHandler()
    }
    memStats("修复版")
}

func main() {
    // 先运行泄漏版,观察 goroutine 数暴涨
    go runLeaky(50)
    time.Sleep(200 * time.Millisecond)
    memStats("泄漏版结束后") // goroutine 数不减!

    // 再运行修复版,观察 goroutine 数回归
    go runFixed(50)
    time.Sleep(200 * time.Millisecond)
}

```

```

memStats("修复版结束后") // goroutine 数恢复基线

// 缓存修复演示
for i := 0; i < 1000; i++ {
    cachePut(fmt.Sprintf("key%d", i%100), i) // 实际只有 100 个键
}
memStats("缓存受控后")

// 现场演示超时退出
timeoutHandler()
memStats("超时退出后")
}

```

设计说明:实验代码把四种知识点织成可对比的形态:泄漏版与修复版并列, `memStats` 在关键节点采样,用 `goroutine` 数与堆对象数两个指标量化差异。验证流程:先跑 `go run main.go` 看两种版本输出差异;再在 `runLeaky` 循环中加入 `time.Sleep` 制造并发,用 `go run -race main.go` 观察竞态;最后把 `handler` 循环提升为 `go test -bench=. -benchmem`,对比 `alloc/op`。扩展:为 `wrongHandler` 写单元测试并用 `goleak` 库自动检测泄漏 `goroutine`(生产团队的标准做法)。

下一章预告:第6章 面向对象。内存搞定了,下一章看"组织代码的方式":Go 没有类!struct + 方法、嵌入(组合优于继承)、隐式接口实现——用 C 的 struct + 函数指针做多态的笨拙,对比 Go 的自然表达。